AI Governance Registry with Deployment Gates

Big Data Systems & Responsible AI Engineering, Topic 32 · Technical report · Code: `README.md` in the repository

1. Problem

A company requires every model to be documented, risk-rated and approved before release. In practice these checks live in review meetings and spreadsheets, so they are easy to skip, and they disagree with what was actually trained. We turn the policy into code: a version-controlled registry of model and dataset cards, a risk classification based on the **EU AI Act** (Regulation (EU) 2024/1689), and CI checks that block a model's deployment when its documentation, lineage, fairness or approval requirements are not met. The system has two guiding principles:

- **The gate verifies, it does not trust.** The risk tier is computed from the use case, not self-declared. Fairness is recomputed from prediction evidence, not read from a card. Lineage is checked against hashes recorded by the training pipeline.
- **The fairness gate is a big data job.** It is written once in Spark, so the check that runs on a 39 k-row hold-out set in CI also audits tens of millions of rows unchanged.

2. Architecture

The registry is plain files in git. Git history then serves as the audit trail, and every change to a card arrives through a pull request.

Artefact	Written by	Content
`registry/models/<id>/card.yaml`	model owner	purpose, users, out-of-scope uses, risk answers, limitations, ethics, human oversight, approval
`registry/models/<id>/run.json`	training pipeline	git commit and dirty flag, sha256 of each training dataset, artifact sha256, metrics
`registry/models/<id>/eval_predictions.parquet`	training pipeline	`y_true`, `y_pred` and the sensitive attributes on the hold-out set: the fairness evidence
`registry/datasets/<id>.yaml`	data steward	datasheet: source URL, licence, source sha256, collection, personal-data statement, known biases
`registry/schemas/*.json`	governance team	JSON Schema per card type; `additionalProperties: false` catches unofficial fields
`registry/policy.yaml`	governance team	required fields, fairness thresholds and approval rule **per risk tier**

`python -m governance.gate [model]` evaluates five gates and exits non-zero if any fails. The GitHub Actions workflow runs it as a **matrix with one job per model**, so each model gets its own required status check. It writes the Markdown report to the job summary and publishes a release artifact only when every gate passes.

3. Risk classification (EU AI Act)

Each card answers a short questionnaire (`risk.use_case`, `interacts_with_people`, `generates_synthetic_content`). `governance/risk.py` maps the answers to a tier:

Tier	Trigger	Examples of `use_case`	Consequence in `policy.yaml`
Unacceptable	Art. 5(1)(a)–(h) prohibited practices	`social_scoring`, `predictive_policing_profiling`, `untargeted_facial_scraping`	never deployable
High	Art. 6(1) Annex I safety components; Art. 6(2) Annex III areas 1–8	`creditworthiness` (5(b)), `employment` (4), `education` (3)	6 extra documentation fields (Arts. 9, 10, 13, 14, 15), fairness gate, independent approval
Limited	Art. 50 transparency	chatbot, synthetic content	`transparency_notice` required
Minimal	none of the above	`other`	base documentation and lineage only

The card also states `declared_tier`. A mismatch is a blocking error, which prevents quietly labelling a credit model "minimal" to skip obligations. For reference, the gates map onto the NIST AI RMF as follows: - the registry implements **GOVERN 1.6** (AI system inventory); - intended-use documentation implements **MAP 1.1**; - the fairness gate implements **MEASURE 2.11**; - the deploy/block decision implements **MANAGE 1.1**.

4. Gates

- **Documentation.** Schema validation, plus the policy's required fields for the computed tier. Empty strings and empty lists count as missing.
- **Lineage.**
- `run.json` must exist.
- The training commit must exist in the repository, and the working tree must have been clean at training time.
- Every training dataset must have a registered card whose `source.sha256` equals the hash recorded at training, and the dataset card must itself pass its schema and required fields.
- The evidence file's sha256 must be unchanged since training, which catches edited predictions.

Hashes are taken of the immutable *source* files, so a re-run on another machine reproduces them. - **Fairness** (high tier). One Spark scan (`groupBy(attributes)` with count, selected, positives and true positives) gives confusion counts per group. Two metrics are derived for each sensitive attribute: - the **disparate impact** ratio (minimum over maximum selection rate), which must be ≥ 0.8 (four-fifths rule); - the **equal-opportunity gap** (maximum minus minimum TPR), which must be ≤ 0.1.

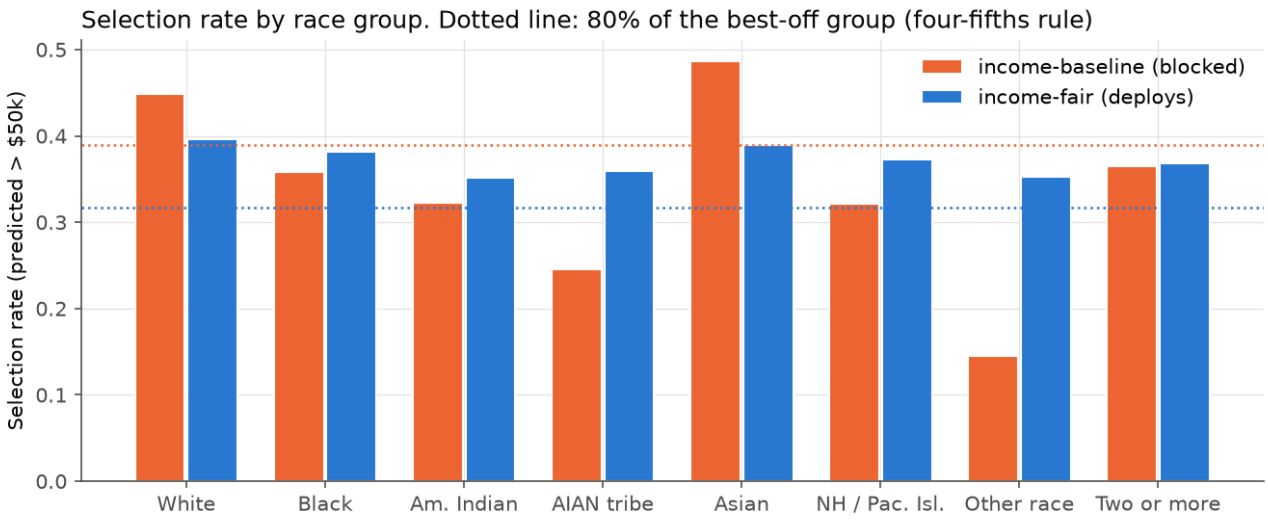
Groups with fewer than 100 rows (or 100 positives, for TPR) are reported but not gated on, because their rates are too noisy. A unit test checks both metrics against fairlearn. - **Approval** (high tier). The status must be `approved` and the approver must differ from the owner (separation of duties).

5. Experiment: three example models

Data. The Folktables ACSIncome task on the 2018 ACS 1-year PUMS for California: 195,665 people, label "income > USD 50 k", sensitive attributes SEX and RAC1P. The national file was not used because of download bandwidth. The 80/20 stratified split leaves a 39,133-person hold-out set. The minimal-risk model uses NYC TLC yellow-taxi trips for January 2023 (3.07 M trips, 600 k sampled).

Models. Scenario: a fictional lender uses an income pre-screen to order credit applications for review (Annex III 5(b), high risk). - `income-baseline`: HistGradientBoosting with SEX and RAC1P removed from the inputs ("fairness through unawareness"), approved by a human reviewer. - `income-fair`: the same inputs, trained with fairlearn's ExponentiatedGradient under a demographic-parity constraint on SEX × RAC1P (bound 0.01). - `taxi-eta`: a trip-duration regressor (minimal risk) whose team skipped parts of the documentation.

Model	Accuracy	DI SEX	DI RAC1P	TPR gap SEX	TPR gap RAC1P	Gate
`income-baseline`	**0.825**	0.802	**0.299**	0.015	**0.331**	❌ fairness
`income-fair`	0.747	0.977	0.885	0.075	0.054	✅ deploy
`taxi-eta`	MAE 2.82 min	n/a	n/a	n/a	n/a	❌ documentation, lineage



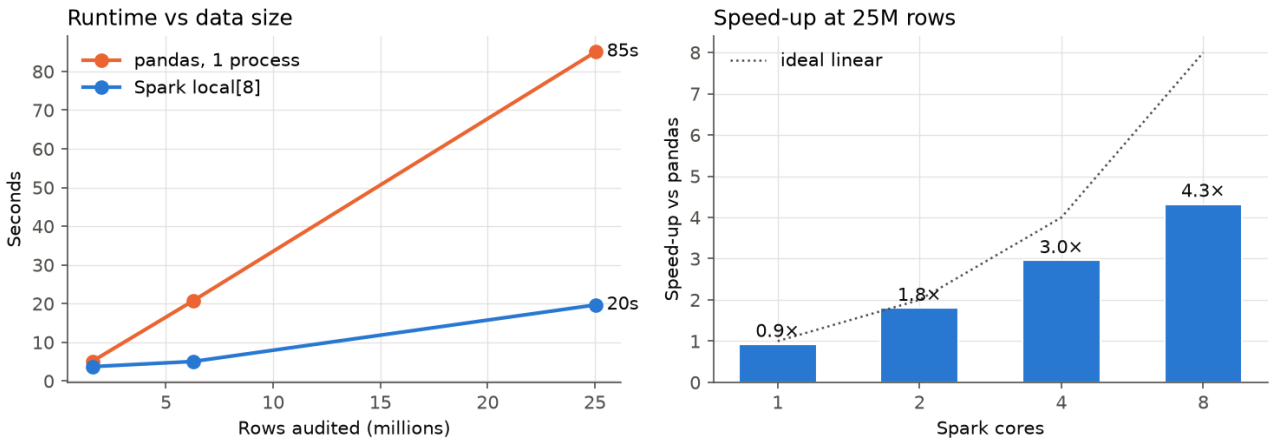
Findings. - **Removing protected attributes did not remove the disparity.** Through proxies such as occupation, place of birth and hours worked, the baseline selects the "Some other race" group at 0.146, against 0.487 for Asian respondents. A human approver signed it off; the automated gate blocked it. - **The constraint fixes the disparity at a cost.** It brings every gated group into a 0.35–0.40 selection-rate band and also halves the TPR gap, at a cost of 7.8 accuracy points. The gate makes that trade-off explicit and recorded instead of implicit. - **`taxi-eta` is blocked for process reasons, not model quality:** an unofficial `known_issues` field instead of `limitations` (a schema error plus a missing field), and a training-data card whose privacy (`personal_data`) statement is still missing. Fairness and approval are `n/a` for its tier, which shows that the requirements scale with risk. - **A tamper test passes.** Rewriting `income-fair`'s predictions after training makes its lineage gate fail (`tests/test_governance.py`).

6. Big data: scaling the fairness audit

Set-up. `pipeline/benchmark_spark.py` replicates the California data with new person ids at ×8, ×32 and ×128, which gives 1.57 M, 6.26 M and 25.0 M rows (16, 64 and 254 MB of Parquet). Each row is scored with the `income-baseline` model through Spark `mapInPandas`, and the gate's own `group_counts` and `metrics` code then computes fairness. - **Baseline:** the same pipeline in single-process pandas. - **Thread count:** one BLAS/OpenMP thread per worker, so the core count means what it says. - **Timing:** best of two runs, with JVM start-up excluded.

Correctness check. Every engine and scale reports the same race disparate impact, 0.2938. This differs from 0.299 in §5 only because here the full dataset is audited, not just the hold-out set.

Rows	pandas (1 core)	Spark local[1]	local[2]	local[4]	local[8]	Spark rows/s (8 cores)
1.57 M	5.1 s				3.7 s	0.42 M
6.26 M	20.7 s				5.1 s	1.24 M
25.0 M	85.1 s	92.0 s	47.2 s	28.8 s	**19.7 s**	1.27 M



Findings. - **pandas scales linearly** at about 0.3 M rows/s, and needs the whole table in memory (1.8 GB of process memory at 25 M rows). - **Spark costs a little on one core.** Spark on one core is 8% slower than pandas, which is the price of the Arrow round-trip. - **Spark gains with more cores.** It reaches 1.8× on 2 cores, 3.0× on 4 and **4.3× on 8** (4.7× against Spark's own single core). Scaling flattens beyond 4 cores because the M4 has 4 performance and 6 efficiency cores. - **Fixed overhead dominates small inputs.** At 1.6 M rows, per-job overhead dominates; throughput only saturates above about 6 M rows. - **The gate is cheap in CI:** 6.2 s for `income-fair` and 4.6 s for `income-baseline`, wall-clock including Spark start-up; `taxi-eta` takes 0.1 s because minimal-risk models never start Spark. - **The gate already processes in batches and streams.** Because the gate is a single aggregation over Arrow batches, it streams the input instead of loading it. The same code would audit production prediction logs on a cluster, which a pandas gate could not do once the logs exceed memory.

7. Limitations

- **Spark ran on one machine.** Spark ran in local mode on one 10-core laptop (M4, 16 GB). The code runs unchanged with `--master spark://…`, but we did not measure a multi-node cluster.
- **The large datasets are replicated.** They are California data replicated with new ids, so they test throughput, not new statistical content. The models are trained and validated on one state and one year only.
- **The risk questionnaire is self-reported.** The gate catches *inconsistent* answers, not dishonest ones; a governance reviewer still owns the questionnaire. The EU AI Act mapping is a simplified encoding of Art. 5, Art. 6/Annex III and Art. 50, not legal advice. Art. 6(3) exemptions and general-purpose AI obligations are not modelled.
- **The fairness checks are narrow.** They are group metrics on two attributes with fixed thresholds. Intersectional groups are reported in the counts but not gated. Demographic parity and equal opportunity can conflict in general.
- **Approval is a YAML field.** A production set-up would also require a CODEOWNERS review by the governance team, or a protected GitHub Environment.
- **The constrained model is randomized** (an ExponentiatedGradient ensemble), so repeated scoring of one person can differ.

8. Ethics statement

All data is public: - **ACS PUMS** is US Census Bureau public-domain microdata, de-identified with disclosure avoidance; we did not attempt re-identification. - **NYC TLC trip records** are NYC open data and contain no rider identities.

Race is special-category data (GDPR Art. 9). It is used only to measure and correct bias, in the spirit of EU AI Act Art. 10(5), and never as a model input. The credit scenario, the teams and the approver identities are

fictional. A model that passes these gates is not thereby "fair": the gates encode a minimum, auditable standard and do not replace domain review or affected-community input.